

## 1. INTRODUCCIÓ

En aquesta pràctica, ens demanaven que duguessin a terme 4 subrutines amb les quals hem hagut de treballar amb el maquinari mapat en memòria, a més de continuar desenvolupant les nostres habilitats en la programació de Easy68K. Les subrutines que ens demanaven que realitzessim son: MAPINIT, MAPRBIT, STR2SEG i BITPOS i a continuació descrivim les seves funcionalitats.

## 2. MAPINIT

En la subrutina següent, rebem com paràmetres d'entrada (D0.B i A0). Els registres modificats han de ser restaurats. Aquí, utilitzarem la instrucció BTST per tal de comprovar els valors dels bits 0,1 del registre D0.

Durant l'execució tendrem 4 situacions diferents(00,01,10,11). Si el bit 0 de menys pes és 1; mostrem la finestra de maquinari, en canvi, si el bit 1 és 1; guardem les adreces de mapatge dins A0.

| Casos possibles | Execució                                                                     |
|-----------------|------------------------------------------------------------------------------|
| D0.B=00         | No es realitza cap operació.                                                 |
| D0.B=01         | Es mostra la finestra de maquinari mapat en memòria.                         |
| D0.B=10         | Es guarden les adreces de mapatge dins A0.                                   |
| D0.B=11         | Es mostra la finestra de maquinari i es guarden les adreces de mapatge a A0. |

Programant aquesta subrutina i executant-la, varem tenir diverses dificultats. Primer de tot, entravem dins un bucle infinit ja que varem introduir la instrucció LEA al principi d'aquest. La condició per a sortir era que el contingut de A1 fos zero. En resum, a l'inici del bucle A1 tornava a apuntar al principi del vector i mai acabava.

## 3. MAPRBIT

La subrutina rep com a paràmetres d'entrada A0 i A1. Tots els registres que s'han modificat han d'ésser restaurats. Suposam que A0 conté les adreces dels botons i A1 les dels visualitzadors de 7 segments. La funcionalitat d'aquesta és mostrar un text

en funció del botó pitjat. Per exemple: si pitjam el botó 4, mostrarem 'FOUR'. Si cap botó està polsat, es mostrarà 'NONE'.

Primer de tot, guardam els registres dins la pila per no modificar el seu contingut previ. Posteriorment, definim el registre D2 com a contador d'un bucle amb 7 iteracions i obtenim dins D0 el nombre del botó pitjat, Dins aquest bucle, anam comprovant quin botó ha estat pitjat mitjançant un CMP D1,D0. D1 va agafant els valors 0,1,2,3... També cal destacar que si surt del bucle sense haver detectat el botó, mostrarem 'NONE'. Després, entrem al mètode DISPLAY, al qual hem fet que A0 apunti al vector de nombres. D1 contindrà el nombre del botó pitjat, registre el qual sirà multiplicat per 8 per tal d'avançar els bytes corresponents i assignar el text en funció del nombre. Finalment, cridarem a STR2SEG per tal de convertir el text a una visualització en 7 segments i restaurarem els valors dels registres utilitzats.

#### 4. STR2SEG

En aquesta subrutina, ens donen com a paràmetres d'entrada els registres A0 i A1. Aquests, apunten al conjunt de caràcters que hem de mostrar per els displays de set segments i la direcció de memòria on hem de colocar aquest conjunt de caràcters, respectivament. A més, tots els registres que utilitzem, hauran d'ésser restaurats al final de la subrutina.

Aquesta subrutina, comprova, un per un, quin tipus de caràcter conté la direcció apuntada per A0 segons el nombre ASCII de cada caràcter. Nosaltres l'hem plantejat de la següent forma: primer de tot, miram si el caràcter és igual a l'espai (osigui, ho serà si el seu codi ASCII es igual a 32). En cas de ésser-ho, restarem a D0 6 (ja que  $32-6=26$ , què és la component exacte del vector on es troba la codificació per als displays de 7 segments del caràcter espai). Si no ho és, mirarem si és un caràcter minúscula. Això ho durem a terme mirant si el codi ASCII del caràcter que estiguem tractant, és major a 96. Si aquesta condició es compleix, restarem a D0 el codi ASCII del caràcter 'a' (97). D'aquesta forma, el contingut de D0 serà la component exacte del vector .CODISSEGM, on es trobarà la codificació de la lletra que hem tractat en aquesta iteració. Finalment, si no és cap dels dos casos anteriors, tindrem a davant un caràcter majúscula. En aquest cas, restarem a D0 el codi ASCII del caràcter 'A' (65). D'aquesta forma, dins D0 quedarà la component exacte del vector .CODISSEGM on es troba la codificació de la lletra tractada. Per tant, a cada iteració, independentment del tipus de caràcter que tinguem davant, al final D0 contindrà l'índex al qual hem d'accedir del vector .CODISSEGM. Aquest, conté totes les codificacions de les lletres de l'abecedari en ordre, deixant la codificació de l'espai per a la component 26.

D'aquesta subrutina, hem tingut complicacions amb la forma d'accedir al vector .CODISSEGM, ja que no sabiem com accedir a una component en concret depenent de la lletra d'entrada. Finalment, ho hem solucionat utilitzant un tipus de direccionament indexat complet per accedir a la component del vector que ens indiqui D0.

## 5. BITPOS

En aquesta subrutina, ens donavem com a paràmetre d'entrada un Byte que haurem d'analitzar (contingut dins D0). Haurem de retornar aquesta mateix registre però que contengui un nombre entre 0-7, indicant la posició del primer bit que contengui un 1. A més, tots els registres utilitzats s'han de restaurar al final de la subrutina.

Per dur-la a terme, primer negam el contingut de D0, ja que llegirem els bits d'esquerra a dreta. D'aquesta forma tindrem tots els bits girats. Una vegada negat, realitzarem un bucle de 7 iteracions on anirem mirant si el bit tractant a cada iteració és igual a 1. En cas de que ho sigui, ficarem dins D0 la posició del bit que estiguem tractant i acabarem la subrutina. En cas de que tots els bits estiguin a 0, ficarem dins D0 un 8, tal i com ens indiquen les instruccions de l'enunciat.

## 6.CONCLUSIONS

En conclusió, l'elaboració d'aquesta metòdica pràctica ens ha resultat molt útil per a expandir els nostres coneixements sobre el 68k, veient funcions no vistes abans com el TRAP, instrucció molt clau en l'àmbit del maquinari o gràfics mateix. També hem repassat coneixement dels curs anterior. Personalment, pensavem que programar en ensamblador com el 68k era una eina molt limitada, però hem canviat totalment d'opinió quan hem vist el conjunt de instruccions que implementa el TRAP, havent la possibilitat d'interactuar amb gràfics i tot. Estam contents amb el resultat i creiem que ha estat una bona pràctica per a iniciar el curs.

Joan, Jaume

